

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5115

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based: 20%

QUESTIONS: 2

Total Marks: 50

Annexure A

Code of Conduct:

*I Nalluri Charan Sai (192372003) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

NCharan Sai

Annexure A Scenario-based

1. Public Key Cryptography for Email Security

An organization plans to implement RSA for secure email communication.

- A. Describe how RSA can be used for encryption and decryption in this scenario.
- B. Explain how key distribution and management play a role in maintaining security.

Explanation: (a) How RSA Can Be Used for Encryption and Decryption

RSA is an asymmetric (public-key) cryptosystem, so every employee in the organization first generates a personal RSA key pair: a public key that is freely shared with anyone who wants to send them a secure email, and a private key that is kept secret and never leaves the employee's own device. This immediately solves the key-distribution problem that plagues symmetric-only email encryption — a sender never needs to have met the recipient beforehand or exchanged a shared secret over a secure channel; they only need the recipient's public key, which by definition is safe to publish openly.

Step 1 — Key generation: Each employee's email client (or a central key-management service) generates an RSA key pair (e, n) as the public key and (d, n) as the private key, following the standard RSA key-generation process — choosing two large primes p and q , computing $n = p \times q$ and $\phi(n) = (p-1)(q-1)$, selecting a public exponent e coprime to $\phi(n)$ (commonly $e = 65537$), and computing the private exponent d as the modular inverse of e modulo $\phi(n)$.

Step 2 — Encryption (confidentiality): Because RSA is computationally expensive on large inputs, it is not used to encrypt the entire email body directly. Instead, the email client generates a random symmetric session key (e.g., for AES), encrypts the actual email content quickly using that symmetric key, and then encrypts only the small session key itself using the recipient's RSA public key: $C_key = (SessionKey)^e \bmod n$. The email is sent as a package containing the AES-encrypted body plus the RSA-encrypted session key. This hybrid approach — exactly how real-world standards such as S/MIME and PGP/GPG operate — combines RSA's key-distribution advantage with the speed of symmetric encryption for bulk data.

Step 3 — Decryption: When the recipient receives the email, they use their own private key d to decrypt the RSA-encrypted session key: $SessionKey = (C_key)^d \bmod n$. Once the session key is recovered, it is used to symmetrically decrypt the actual email body. Because only the intended recipient possesses the matching private key d for the public key e that was used to encrypt the session key, no other party — including the email server operators relaying the message — can recover the session key or read the email content.

Step 4 — Digital signatures (authenticity and integrity): RSA is also used in the reverse direction to provide authentication and non-repudiation, which confidentiality alone does not guarantee. The sender computes a cryptographic hash (message digest) of the email content, then encrypts that hash with their own private key to produce a digital signature: $Signature = Hash(Message)^d_{sender} \bmod n$. The recipient independently computes the hash of the received message and decrypts the received signature using the sender's public key; if the two match, the recipient has strong assurance that the email genuinely came from the claimed sender (authenticity) and was not altered in transit (integrity), since only the sender's private key could have produced a signature that verifies correctly under the sender's public key.

(b) Role of Key Distribution and Management in Maintaining Security

RSA's security guarantees only hold if the public keys used are genuinely tied to the correct identities and if private keys remain truly secret; both of these properties depend entirely on how the organization distributes and manages keys, not on the RSA mathematics itself. Several concerns must be addressed:

1. Authenticity of public keys (avoiding impersonation): If an attacker can trick a sender into using the attacker's public key instead of the real recipient's — a classic man-in-the-middle substitution — the attacker can intercept and read messages the sender believes are securely addressed to someone else. The organization must therefore bind each employee's public key to their verified identity through a trusted mechanism, most commonly a Public Key Infrastructure (PKI): a Certificate Authority (CA) issues a digital certificate that cryptographically binds a public key to an employee's name/email address, and this certificate is itself signed by the CA's private key so that anyone can verify it using the CA's well-known public key. This is the model used by S/MIME. An alternative, decentralized model used by PGP/GPG is the 'web of trust', where users sign each other's keys to vouch for their authenticity without relying on a central CA — suitable for smaller or less hierarchical organizations.

2. Secure storage of private keys: Since anyone who obtains an employee's private key can read all email ever encrypted to that employee and forge signatures in their name, private keys must be protected at least as strongly as the data they secure — typically stored in an encrypted local keystore protected by a passphrase, or, for higher-value keys (such as the organization's CA root key), in a Hardware Security Module (HSM) that never exposes the raw key material even to the systems that use it.

3. Key revocation and expiry: Employees leave the organization, devices are lost or stolen, and private keys are occasionally compromised. The key-management system must support revoking a public key (so others stop trusting it and stop encrypting new messages to it) via mechanisms such as a Certificate Revocation List (CRL) or the Online Certificate Status Protocol (OCSP), and should enforce periodic key expiry and rotation so that any undetected compromise has a bounded window of impact rather than remaining valid indefinitely.

4. Scalability of key distribution: As the organization grows, manually verifying and exchanging public keys between every pair of employees becomes impractical. A centrally managed directory service (an internal key server, or an LDAP/Active Directory integration publishing employee certificates) allows any employee's email client to automatically look up and retrieve the current, valid public key for any colleague without manual key exchange, which is essential for the system to remain usable at organizational scale.

5. Lack of forward secrecy in plain RSA key transport: A subtle but important limitation is that if RSA is used directly to encrypt session keys (as described above) and an employee's private key is ever compromised, an attacker who has been passively recording past encrypted emails could decrypt every session key — and therefore every email — ever sent to that employee, even ones sent years earlier. This is because the same static RSA private key protects every past session key. Organizations concerned about this risk increasingly prefer protocols that combine RSA (for authentication) with an ephemeral Diffie-Hellman key exchange (for the actual session key), so that each session key is independently random and not recoverable even if the long-term RSA private key is later compromised — a property known as forward secrecy.

In summary, RSA supplies the mathematical mechanism for confidentiality (via hybrid encryption) and authenticity (via digital signatures) in the email system, but the

organization's overall security depends just as heavily on the surrounding key-management infrastructure — certificate issuance and verification, secure private-key storage, revocation, scalable distribution, and awareness of RSA's forward-secrecy limitations — as it does on the RSA algorithm itself.

2. Cryptography for IoT Devices

A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

- A. Analyze the constraints of IoT devices.
- B. Recommend a suitable cryptographic approach and justify your answer.

Explanation: (a) Analysis of IoT Device Constraints

Smart-home IoT devices such as door sensors, smart plugs, thermostats, and light switches are fundamentally different from servers or even smartphones, and these differences directly shape which cryptographic approach is practical:

- Limited computational power: Most IoT devices use low-cost microcontrollers (MCUs) with clock speeds of only tens of MHz and no dedicated cryptographic co-processor, so any cryptographic operation must complete using very modest general-purpose compute capability.
- Limited memory (RAM and flash storage): IoT MCUs frequently have only tens to a few hundred kilobytes of RAM and similarly constrained flash storage, leaving little room for large cryptographic keys, certificates, or bulky algorithm implementations alongside the device's actual application firmware.
- Limited energy / battery life: Many smart-home devices run on small batteries or energy-harvesting sources and are expected to operate for months or years without recharging; every millijoule spent on cryptographic computation and on transmitting larger keys/certificates over the radio directly shortens battery life.
- Limited network bandwidth: IoT devices typically use low-power wireless protocols (e.g., Zigbee, Bluetooth Low Energy, Thread, or constrained Wi-Fi) with much lower throughput than a broadband connection, making the size of exchanged keys and certificates a meaningful factor in both latency and energy cost, since radio transmission is often the single most power-hungry operation a device performs.
- Cost sensitivity at scale: A smart-home ecosystem may involve dozens of individually inexpensive devices; adding a more powerful (and more expensive) processor purely to accommodate heavier cryptography conflicts with the low per-unit cost that makes IoT deployments commercially viable.
- Physical exposure: Unlike a server in a locked data centre, smart-home devices are physically accessible to anyone in or near the home, increasing the risk of physical tampering or side-channel attacks, and reinforcing the need for efficient cryptography that can still be implemented with side-channel protections despite constrained resources.
- Need to scale key management across many devices: A single smart-home hub may need to securely provision, authenticate, and communicate with dozens of individual devices, so the chosen scheme's key sizes and computational cost are multiplied across every device and every connection, not just a single link.

(b) Comparing RSA and ECC for This Scenario

RSA and ECC are both public-key approaches capable of providing the confidentiality (via key exchange/encryption) and authenticity (via digital signatures) the smart-home system needs, but they differ enormously in the resources required to reach a given security level, because they rely on different underlying hard mathematical problems: RSA's security rests on the difficulty of factoring large integers, while ECC's rests on the difficulty of the elliptic-curve discrete logarithm problem, which is significantly harder per bit of key size. This means ECC can match RSA's security using much smaller keys:

RSA Key Size (bits)	Equivalent ECC Key Size (bits)	Approximate Security Level
1024	160	80-bit (legacy / weak by modern standards)
2048	224	112-bit
3072	256	128-bit (current baseline recommendation)
7680	384	192-bit
15360	521	256-bit

At the commonly recommended 128-bit security level, RSA requires a 3072-bit key while ECC achieves the same strength with only a 256-bit key — roughly a 12-fold reduction in key size. This difference compounds directly into every constraint identified above:

Factor	RSA (3072-bit, ~128-bit security)	ECC (256-bit, ~128-bit security)
Key/signature size	Large (384+ bytes)	Small (~32–64 bytes)
Computation cost (signing/key exchange)	Higher — large modular exponentiations	Lower — smaller-field elliptic-curve point operations
Memory footprint	Larger code + key storage	Significantly smaller
Energy per operation	Higher	Substantially lower
Bandwidth for handshake/certificates	Higher	Lower — smaller keys and signatures to transmit

(c) Recommendation and Justification

Recommendation: Elliptic Curve Cryptography (ECC) — specifically a well-vetted curve such as Curve25519 for key exchange (via ECDH/X25519) and a corresponding elliptic-curve signature scheme such as ECDSA or EdDSA for authentication — is the more suitable cryptographic approach for this smart-home IoT scenario.

This recommendation follows directly from the constraint analysis in part (a): every one of the identified limitations (limited computation, limited memory, limited energy, limited bandwidth, and cost sensitivity) is directly alleviated by ECC's dramatically smaller key sizes for equivalent security, as quantified in the comparison tables above. A door sensor or smart plug performing an ECC key exchange or signature verification will consume less battery, need less RAM and flash to store keys/certificates and hold the algorithm implementation, transmit fewer bytes over its constrained wireless link (saving further energy, since radio transmission is typically the most power-expensive operation on such devices), and can be manufactured with a cheaper, less powerful microcontroller than an equivalent RSA-based design would require — all while

achieving the same cryptographic strength.

This trade-off is precisely why ECC (rather than RSA) underlies most modern IoT and embedded security standards, including Bluetooth Low Energy's secure pairing, Thread/Matter (the current smart-home interoperability standard, which mandates ECC-based certificates and key exchange), and TLS 1.3's preference for ECDHE key exchange over classical RSA key transport. RSA remains a reasonable choice in less resource-constrained parts of the system — for example, the smart-home hub or a cloud backend with ample compute and power available, where RSA's simplicity and extremely wide legacy compatibility can still be a practical advantage — but for the battery-powered, low-power end devices described in this scenario, ECC is the clearly justified choice.

One caveat worth noting: ECC implementations must still be carefully engineered to resist side-channel attacks (timing or power-analysis attacks that could leak the private key during point-multiplication operations), which is a meaningful engineering concern given the physical accessibility of smart-home devices noted in part (a); this is a standard, well-understood challenge in embedded ECC implementations and does not change the overall recommendation, but it should be an explicit requirement placed on whichever ECC library or hardware the organization selects.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

